

Offen statt teuer: Wie Open-File-Formate Kosten senken und Innovation beschleunigen

Offene Dateiformate wie Parquet entkoppeln Daten von traditionellen SAS7bdat-Files und machen Storage, Analytics und KI deutlich effizienter. Mit der neuen DuckDB-Engine in Viya kann man mit etwas SAS Base Code idealerweise den TCO reduzieren, Performance gewinnen und gleichzeitig seine Datenstrategie in hybriden Architekturen zukunftssicher aufstellen.

Michael Herrmann, Solutions Architect SAS Deutschland / 20.5.2026





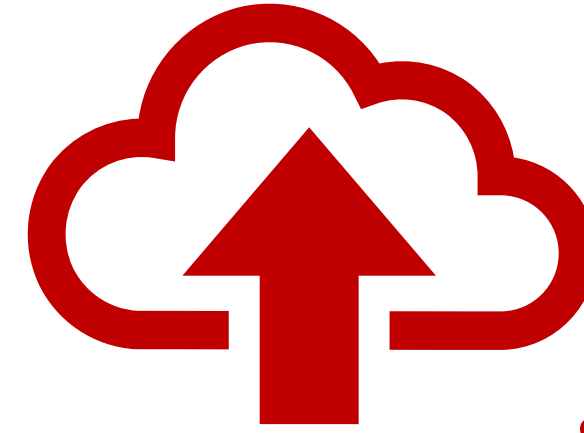
Preis



Performance



Offenheit (Extensibility)



... ab in die
Cloud !?

„50* Jahre sas7bdat“ ;)

Kosten optimieren beim Speichern, alternativ zu SAS Datasets?

EINERSEITS

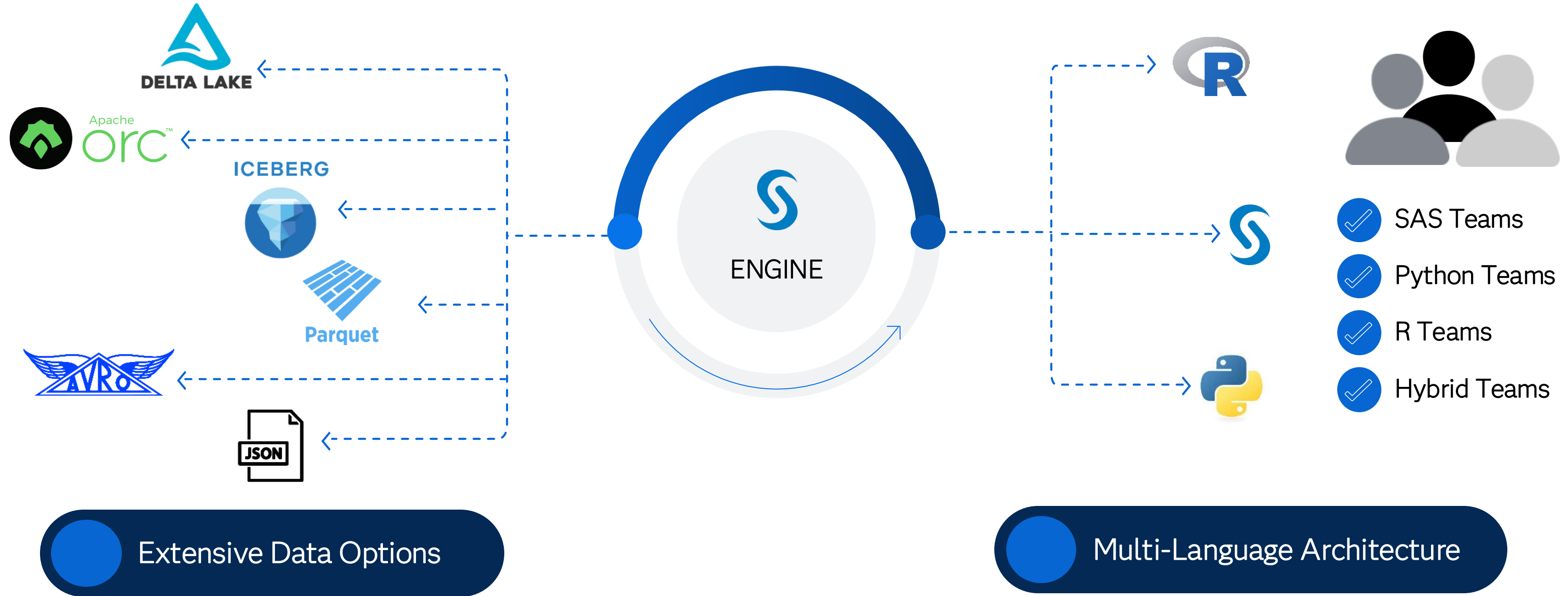
- robust & treiberlos
- default (in SAS)
- MVA/ CEDA/ CPort ...
- hoch optimierter Data-Step/ Procs
- langzeit kompatibel
- Compress, Index, GDG
- ... halt ein File im Folder

ANDERERSEITS

- liegt „breit“ auf der Platte
- braucht SAS Compute zum Zugreifen, wenig interoperabel
- kein Transaktionsformat
- Formate prima, aber halt außerhalb
- Encoding, SPDS, SPDE, sasHdat ...
- ... proprietär (okay für SAS'ler – gewöhnungsbedürftig für den Rest)

*) Transparenzhinweis: real knappe 28 Jahre, SAS 7.0 „The Nashville Release“ vom 1. Oktober 1998

Extensibility?



2026: SAS ist Teil eines analytischen Ökosystems

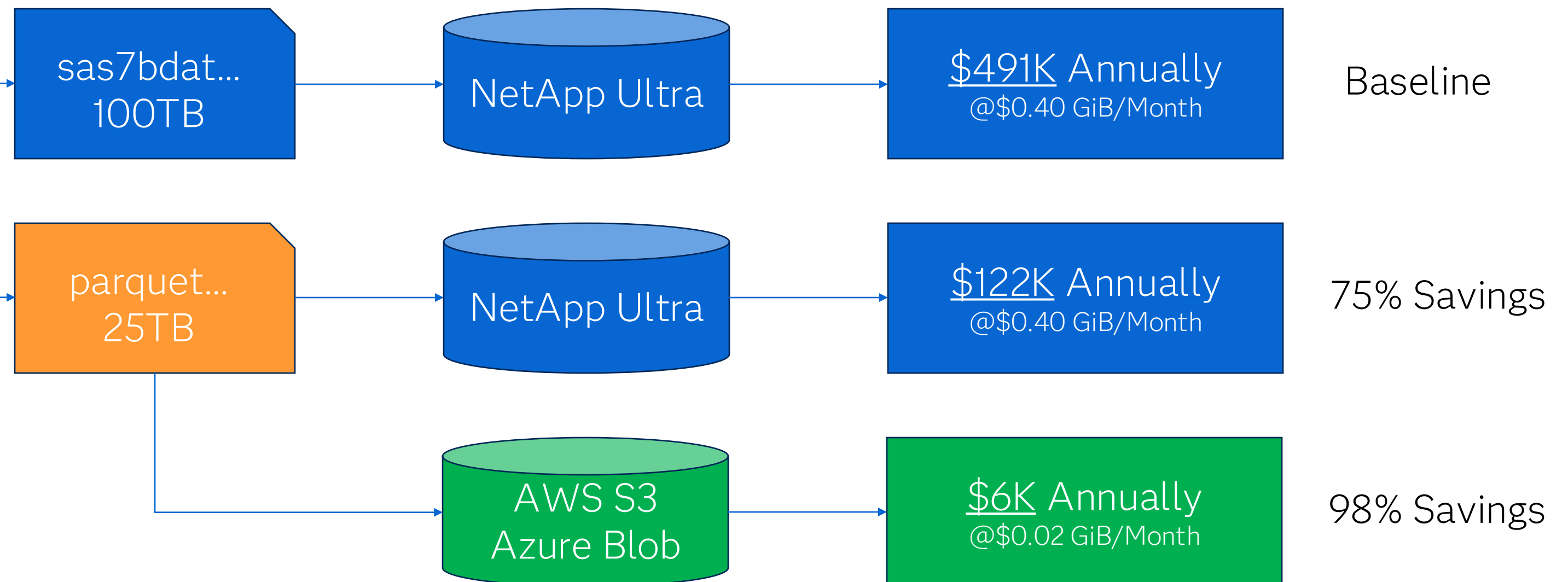
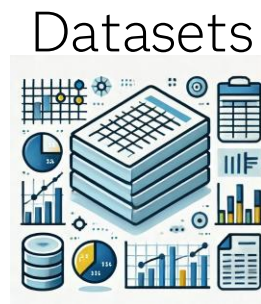
... und *hat* reagiert auf die aktuellen Markttrends

- Der (traditionelle) Analytics-Stack verschiebt sich rasant zu
 - quelloffenen Programmiersprachen wie Python oder R (oder LLM-Prompts ;)
 - Industriestandards wie GIT, Kubernetes, VS Code, API-first etc.
 - offenen Formaten wie Parquet: wegen Kosten, Datenwachstum und Cloud-Bedarf.
- **Denn Storage-Effizienz wird in Cloud-Projekten sichtbar und verursacht Budgetdruck bei großen Datensätzen, auch on-premises, gern „von oben“.**
- Ein „als zu *hoch* empfundener“ TCO bremst schlimmstenfalls Innovationen, Modernisierungen (zu bspw. Viya) und trübt die Sicht auf den real hohen Wertbeitrag Ihrer SAS-Umgebungen für den Unternehmenserfolg – und die Chance auf „mehr Analytics, Decisioning & KI“ für Ihre Anwender

→ daher: reden wir mal über offene Storage-Alternativen!

Speicherkosten optimieren: „kalte“ Daten in Parquet

Potentieller Gewinn beim Speichern als “Open File Format” per Parquet-Libname



SAS/Access to DuckDB

Für smarte SQL – *jenseits* eines „nur-Speichern-unter“ im Data-Step



DuckDB ist eine Open-Source-Analyse-Engine, die direkt im SAS Compute läuft



Vektorisierte, optimierte Abfrageausführung auf spaltenbasiertem Speicher



Erfordert keine Serverinstallation und keine externen Abhängigkeiten



Ab Version 2025.07 ist DuckDB als Teil von SAS Viya immer (!) enthalten



Umfassende Unterstützung für viele offene Datei- und Tabellenformate*

*) neben Parquet (heutiges Beispiel) auch Avro, CSV, JSON, Iceberg, Delta ... mit „gemischten“ Joins

Performance Benchmark

TPC-H: (22) Decision Support Queries

The TPC-H is a decision support benchmark. This benchmark illustrates decision support systems that examine large volumes of data, execute queries with a high degree of complexity, and give answers to critical business questions.

11.8 hours for SAS BASE Libname engine & SAS7BDAT
vs
20.2 seconds with SAS/ACCESS to DuckDB & Parquet

Performance improvement:

- Average: 2000x
- Median: 4.8x

SQL source code : <https://github.com/dragansah/tpch-dbgen/blob/master/README>

Scale Factor = 1 GB. Tests were executed on 16 vCPU SAS Container on NetApp Premium

= 11.8 Hrs+

Query	BASE Engine SAS code	DuckDB SAS code	DuckDB Native SQL
Source	SAS7BDAT	Parquet	Parquet
Q01	2.99	0.51	0.2
Q02	224.21	0.64	0.12
Q03	3.7	0.57	0.16
Q04	17257	0.52	0.13
Q05	4.03	0.98	0.17
Q06	2.49	0.32	0.11
Q07	3.93	0.79	0.17
Q08	3.87	0.8	0.18
Q09	4.74	0.98	0.23
Q10	3.68	1	0.26
Q11	1.13	0.51	0.09
Q12	3.42	0.66	0.15
Q13	1.22	0.28	0.14
Q14	2.62	0.38	0.12
Q15	2.94	0.8	0.1
Q16	6.09	5.48	0.12
Q17	37.41	0.67	0.13
Q18	4.37	1.08	0.22
Q19	22886	0.45	0.19
Q20	1842	0.85	0.14
Q21	ERR	1.49	0.33
Q22	311	0.44	0.09
Total	42408.84	20.2	3.55



Beispiel: Duck-DB im SAS Code

Lesen und Schreiben „wie sonst auch“ (nur ein Beispiel für x Varianten)

```
/* 1. DuckDB-Library anlegen */
libname duck DUCKDB
    database="&MYHOME/cars.duckdb";

/* 2. SASHELP.CARS in DuckDB-Tabelle kopieren */
proc sql;
    create table duck.cars as
    select *
    from sashelp.cars;
quit;

/* 3. DuckDB SQL ausführen: Tabelle als Parquet exportieren */
proc sql;
    connect using duck;
    execute (
        COPY cars
        TO "&MYHOME/cars.parquet"
        (FORMAT PARQUET)
    ) by duck;
    disconnect from duck;
quit;
```

```
/* 4. Parquet-Datei in eine DuckDB-Tabelle laden */
proc sql;
    connect using duck;
    execute (
        CREATE OR REPLACE TABLE cars_from_parquet AS
        SELECT *
        FROM read_parquet("&MYHOME/cars.parquet")
        WHERE Origin = 'Europe'
    ) by duck;
    disconnect from duck;
quit;

/* 5. DuckDB-Tabelle nach SAS WORK kopieren */
data work.cars_from_parquet;
    set duck.cars_from_parquet;
run;

/* 6. Alternative: File direkt per Parquet-Libname lesen */
libname pq PARQUET "&MYHOME";
data work.cars;
    set pq.cars;
run;
```

Mögliche „Nebenwirkungen“ mit Parquet

... denn Open File Formats sind kein Allheilmittel, sondern wirken spezifisch

- 1) SAS Formate, Labels ... knifflig
- 2) Gut für *große* Datasets >250 MB (Metadata-Overhead, I/O)
- 3) Besser im *Lesen* als im „klein-klein-Schreiben“
- 4) *Spaltenorientierte* Speicherung beachten = ineffizient beim „*alle-Zeilen-Lesen*“
- 5) Binäres Format (wie sas7bdat), also kein CSV/JSON
- 6) Partitionierung per Unterordner/files = gewissenhaft nutzen →
- 7) Parquet unterstützt „Schema evolution“ (neue Spalten, alte raus), aber Obacht
- 8) Parquet hat keine „built-in ACID (Atomicity, Consistency, Isolation, Durability) transaction guarantees“ = daher Apache Iceberg, oder Delta Lake betrachten.

Beispiel mit Partitionierung:

```
/data/sales/  
  year=2025/  
    month=01/  
      part-0001.parquet  
      part-0002.parquet  
    month=02/  
      part-0001.parquet  
  year=2026/  
    month=01/  
      part-0001.parquet
```

[Clear selection \(17\)](#)

- ☒ AWS Redshift
- ☒ CSV
- ☒ CSV - Compressed
- ☒ Google BigQuery
- ☒ ORC
- ☒ Parquet - Compress=brrotli
- ☒ Parquet - Compress=Default
- ☒ Parquet - Compress=lz4
- ☒ Parquet - Compress=snappy
- ☒ Parquet - Compress=zstd
- ☒ SAS Dataset - SAS7BDAT
- ☒ SAS Dataset - SAS7BDAT Compressed
- ☒ SAS Dataset - SAS7BDAT Gzip
- ☐ SAS Dataset - SASHDAT
- ☐ SAS Dataset - SASHDAT Compressed
- ☒ SAS Dataset - SPDE
- ☒ SAS Dataset - SPDE Compressed
- ☒ Singlestore
- ☒ Snowflake

Location

[Select all](#)

- ☐ AWS
- ☐ Azure Blob Storage
- ☐ Azure Datalake Storage
- ☐ Cloud
- ☐ Co-located Singlestore
- ☐ Google Cloud
- ☐ Google Cloud Storage
- ☐ Local File System
- ☐ S3

	Dataset Format	Location	Size in GB	Space Savings	Cost reduction	Get (Seconds)	Put (seconds)	Cost (Yearly)	Cost (monthly)	Get (Rows per minute)	Put (Rows per minute)
21	ORC	Azure Blob Storage	0.01	99.13%	99.96%	4.46	9.32	0.00	0.00	116M	
22	ORC	S3	0.01	99.18%	99.96%	4.12	8.26	0.00	0.00	126M	
23	ORC	Azure Datalake Storage	0.01	99.13%	99.96%	4.87	10.08	0.00	0.00	107M	
24	ORC	Local File System	0.00	99.31%	99.52%	3.95	5.59	0.02	0.00	132M	
25	SAS Dataset - SAS7BDAT Gzip	Azure Datalake Storage	0.00	99.59%	99.97%	3.58	7.90	0.00	0.00	145M	
26	SAS Dataset - SAS7BDAT Gzip	Local File System	0.00	99.59%	99.62%	3.52	7.89	0.01	0.00	147M	
27	SAS Dataset - SAS7BDAT Gzip	S3	0.00	99.59%	99.97%	3.62	6.79	0.00	0.00	143M	
28	SAS Dataset - SAS7BDAT Gzip	Azure Blob Storage	0.00	99.59%	99.97%	3.66	7.51	0.00	0.00	142M	
29	CSV - Compressed	S3	0.00	99.73%	99.99%	5.02	22.78	0.00	0.00	104M	
30	CSV - Compressed	Azure Datalake Storage	0.00	99.79%	99.99%	5.44	24.46	0.00	0.00	96M	
31	CSV - Compressed	Azure Blob Storage	0.00	99.79%	99.99%	5.09	24.03	0.00	0.00	102M	
32	CSV - Compressed	Local File System	0.00	99.79%	99.80%	4.68	23.07	0.01	0.00	111M	
33	Parquet - Compress=Default	S3	0.00	99.97%	100.00%	1.58	8.84	0.00	0.00	329M	
34	Parquet - Compress=Default	Azure Datalake Storage	0.00	99.97%	100.00%	0.24	9.63	0.00	0.00	2.2B	
35	Parquet - Compress=Default	Azure Blob Storage	0.00	99.97%	100.00%	7.40	9.36	0.00	0.00	70M	
36	Parquet - Compress=Default	Local File System	0.00	99.97%	99.98%	7.45	8.05	0.00	0.00	70M	

Size in GB by Dataset Format grouped by Location



Dataset Management made easy

Help users adopt your data choices in your daily jobs runs

SAS Data Macros

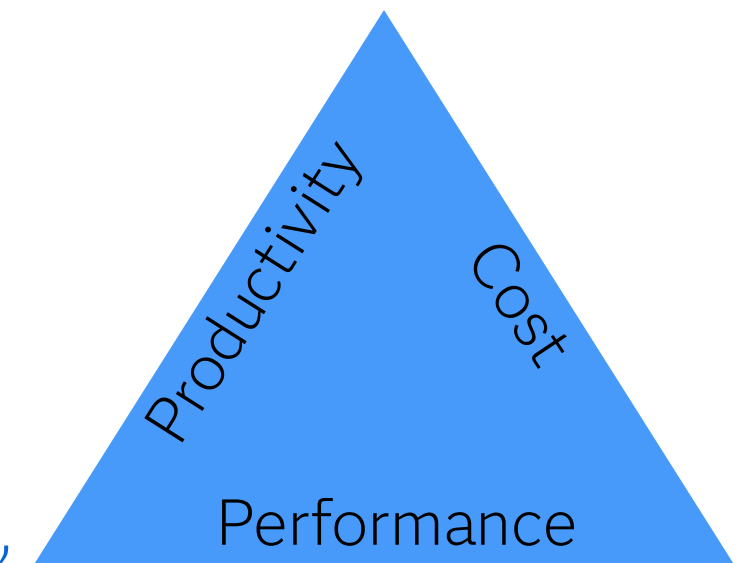
Implement your dataset management strategy

- Reduce dataset size early
- Compress datasets
- Dataset Tiering
- Cache active datasets in-memory or on local disk

Fertige Makros zum
Umkopieren im
Projekt, testhalber
oder produktiv

“Easily adapt to new dataset storage choices using efficient techniques”

Making the right choices helps reduce storage costs, increase performance, and make users more productive



DuckDB: Was leistet diese Engine in SAS Viya?



Performance & Kosteneffizienz

DuckDB ist für analytische Workloads optimiert und kann große Datenmengen schnell im Arbeitsspeicher verarbeiten. Dadurch sinkt der Bedarf an teurer Infrastruktur und die Abfrage-Performance verbessert sich deutlich.



Unterstützung offener Dateiformate

DuckDB unterstützt das Lesen von Parquet, JSON, CSV sowie auch Iceberg und Delta Lake (aktuell read-only). Dadurch eignet sich die Lösung ideal für moderne Data-Lake-Architekturen.



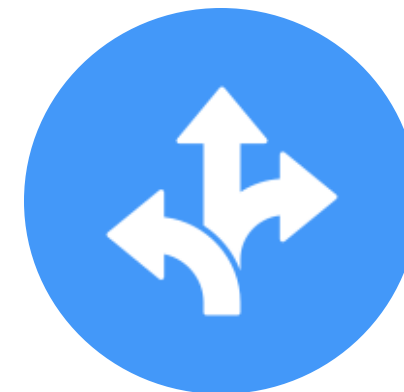
SAS-Integration

Mit der neuen LIBNAME-Engine und der Unterstützung von PROC SQL können SAS-Anwender DuckDB wie jedes andere Datenbanksystem verwenden. Das ermöglicht nahtlose Workflows und reduziert die Lernkurve erheblich.



Data Federation

Abfragen über mehrere Datenquellen hinweg (Dateien, Datenbanken, Cloud-Storage etc.) sind möglich – als würden diese eine einzige Datenbank bilden. Das funktioniert ohne vorheriges Verschieben oder Kopieren der Daten.



Flexibilität

DuckDB kann sowohl rein im Arbeitsspeicher als auch mit persistenter Speicherung betrieben werden. Zudem unterstützt es Cloud-Objektspeicher wie S3 oder Azure Blob Storage über sichere Authentifizierungsmechanismen.



Zukunftssicherheit

Durch Erweiterungen ist DuckDB offen für neue Technologien. Da sich Datenökosysteme zunehmend in Richtung offene, portable und cloud-native Formate entwickeln, trägt DuckDB dazu bei, dass SAS wettbewerbsfähig und interoperabel bleibt.

Nützliche Links für „eigene Experimente“

- 1) SAS Dokumentation „SAS/ACCESS Interface to DuckDB“:
https://go.documentation.sas.com/doc/en/pgmsascdc/v_074/acwn/p1ozr0t2ly4bc2n0zxncjtshlyor.htm
- 2) SAS Dokumentation „Working with Parquet Files“:
https://go.documentation.sas.com/doc/en/pgmsascdc/v_074/acrelb/p0f6u0302oo31cn1ichoxv9rif1i.htm
- 3) Training „Working with DuckDB in SAS Viya“ (Learn.SAS):
<https://learn.sas.com/course/view.php?id=8699>
- 4) SAS Communities Serie zum Einlesen (Blog):
<https://communities.sas.com/t5/SAS-Communities-Library/SAS-DuckDB-Series-Using-the-DuckDB-Engine/ta-p/984095>
- 5) Case Study mit Vergleichszahlen (Blog):
<https://communities.sas.com/t5/SAS-Communities-Library/Speeding-Up-Workloads-with-Ducks-Case-Study-of-New-DuckDB-ACCESS/ta-p/977090>
- 6) „Performant Aggregations on Parquet using DuckDB“ (Blog):
<https://communities.sas.com/t5/SAS-Communities-Library/Performant-Aggregations-on-Parquet-using-DuckDB/ta-p/981117>
- 7) Iceberg-Support mit DuckDB (Blog):
<https://blogs.sas.com/content/sgf/2026/02/23/sas-duckdb-iceberg>

Ideen, Fragen & Wünsche?

Michael.Herrmann@sas.com

